

Deep Learning

Aula 01 · Fundamentos e Redes Neurais

Da Intuição Biológica ao Aprendizado de Máquina



Por que estamos aqui?

Aprendizado baseado em problema real



Problema de Negócio

Uma operadora de telecom perde 22% dos clientes todo mês (churn). Cada cliente perdido = R\$1.200 de receita/ano. Com 500k clientes, são R\$132M em risco anual. Modelos de regras falharam. Regressão logística chegou a 67% de acurácia. Precisamos de algo mais poderoso.



22%

Churn Mensal



R\$132M

Receita em Risco



67%

Melhor Modelo Atual

Deep Learning: onde está e onde chega

Aplicações reais de mercado



Bancos & Fintech

Detecção de fraude em tempo real
(Nubank, Itaú)



Saúde

Diagnóstico por imagem
(Einstein, Hospital Sírio)



Mobilidade

Reconhecimento de padrões
(99, Uber)



Varejo & E-commerce

Recomendação de produtos
(Mercado Livre, Amazon)



Telecom

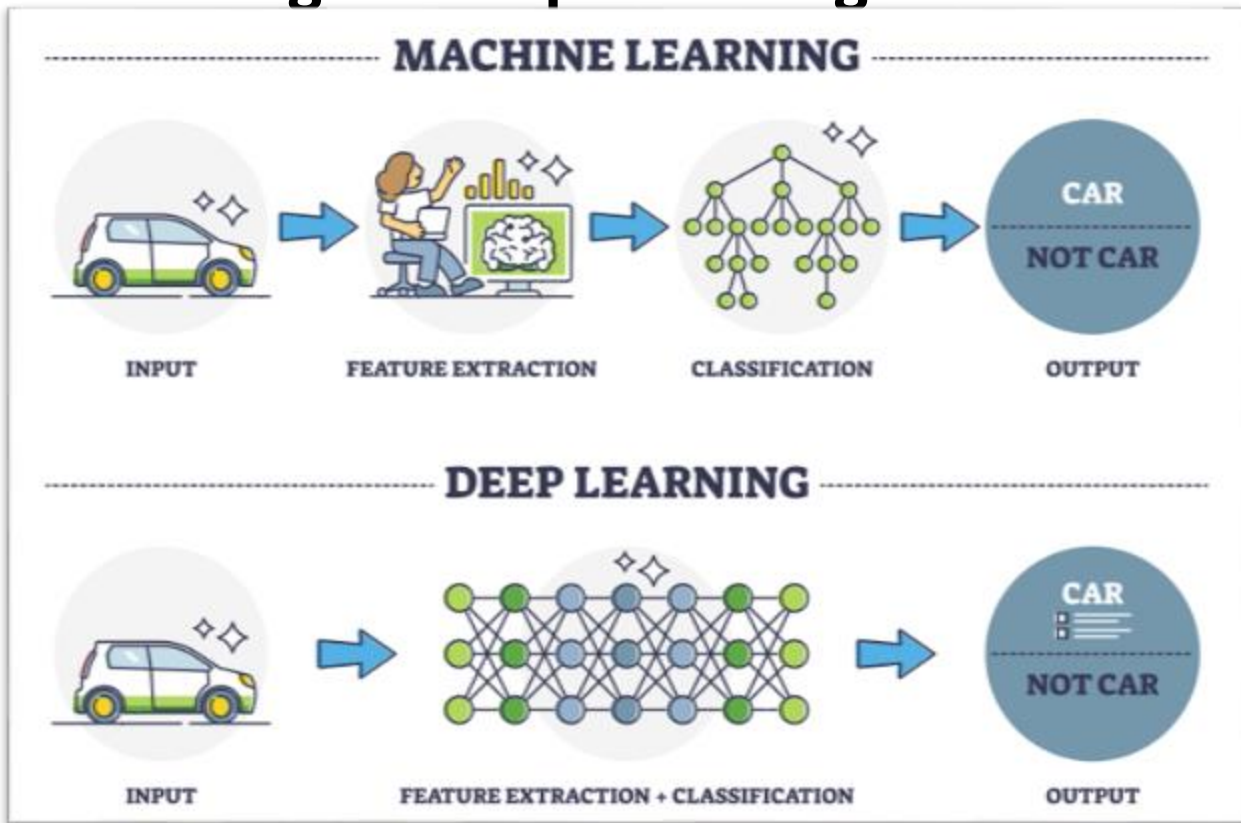
Previsão de churn e NPS
(nosso problema!)



LLMs & GenAI

ChatGPT, Claude, Gemini
(base: transformers + DL)

Machine Learning vs Deep Learning

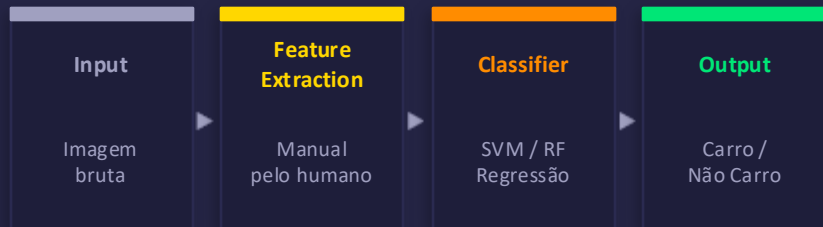


Machine Learning vs Deep Learning

O que muda com a profundidade?



Machine Learning



⚠ **Engenharia de features** é feita manualmente por especialistas de domínio

Limitações:

- ✗ Features ruins → modelo ruim (GIGO)
- ✗ Não escala bem com volume de dados
- ✗ Requer expertise manual por domínio

VS



Deep Learning



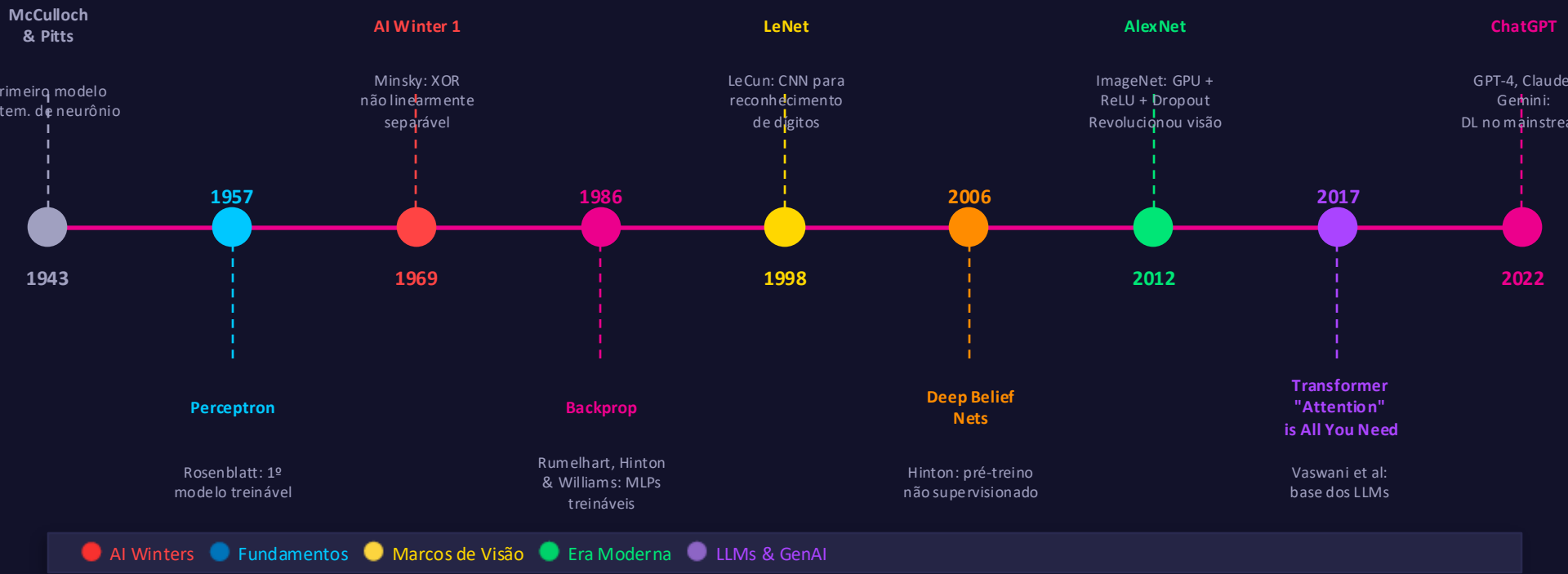
Rede Neural Profunda

Vantagens:

- ✓ Aprende features automaticamente
- ✓ Escala com volume de dados
- ✓ State-of-the-art em imagem, voz, texto

Linha do Tempo: Redes Neurais & Deep Learning

De McCulloch & Pitts (1943) ao ChatGPT (2022)



O Neurônio: da Biologia ao Código

Neurônio Biológico

- Dendritos recebem sinais
- Corpo celular soma e processa
- Axônio transmite o sinal
- Sinapses conectam neurônios
- Dispara se limiar é atingido

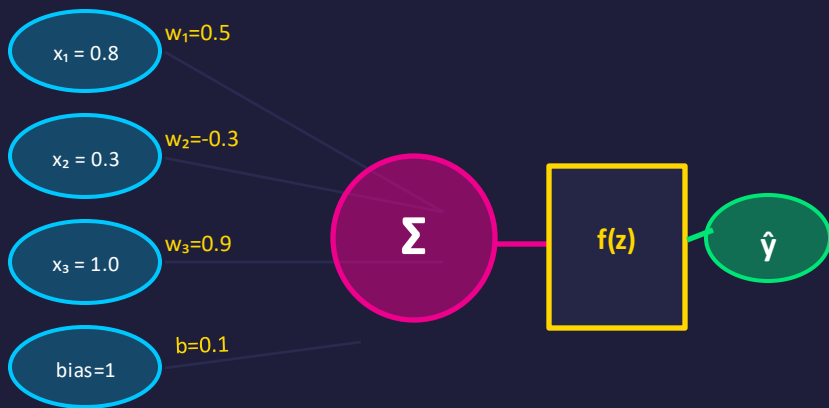
Neurônio Artificial



<i>Dendritos</i>	Entradas x_1, x_2, x_3
<i>Peso sinapse</i>	Pesos w_1, w_2, w_3
<i>Corpo celular</i>	Soma ponderada $\sum(w_i x_i) + b$
<i>Limiar</i>	Função de ativação $f(z)$
<i>Axônio</i>	Saída $\hat{y}$

Perceptron: Função Degrau & Atualização de Pesos

Arquitetura do Perceptron



$$z = 0.8 \times 0.5 + 0.3 \times (-0.3) + 1.0 \times 0.9 + 0.1 = 1.21$$

Função Degrau

$$f(z) = \begin{cases} 1 & \text{se } z \geq 0 \\ 0 & \text{se } z < 0 \end{cases}$$

$$\begin{aligned} z < 0 &\rightarrow \hat{y} = 0 \text{ (Não Churn)} \\ z \geq 0 &\rightarrow \hat{y} = 1 \text{ (Churn!)} \end{aligned}$$

$$z = 1.21 \geq 0 \rightarrow \hat{y} = 1$$

✓ **CHURN previsto!**

Regra de Atualização

$$y_{\text{real}} = 1$$

Real (supervisionado)

$$\hat{y} = 1$$

Predição atual

$$e = y - \hat{y} = 0$$

Erro (acertou!)

$$\Delta w = \eta \times e \times x$$

$$\eta = 0.1 \text{ (lr)}$$

$$w \leftarrow w + \Delta w$$

Atualizar pesos

Só atualiza se errou!

Perceptron: o bloco mais simples

Frank Rosenblatt, 1957 — o primeiro modelo treinável

Formulação Matemática

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

$$\hat{y} = 1 \text{ se } z \geq 0 \quad | \quad \hat{y} = 0 \text{ se } z < 0$$

Exemplo: Aprovação de Crédito

x_1 = Score de crédito x_2 = Renda mensal

x_3 = Histórico de pagamentos

→ Saída: Aprovar (1) ou Negar (0)



Limitação Crítica



O Perceptron simples só consegue separar dados linearmente separáveis.

Na telecom: clientes com alto uso de dados e baixo saldo em conta podem tanto churnar quanto não. Uma linha reta não separa isso!

→ Precisamos de **MÚLTIPLAS camadas**

Combinando Regiões Não Lineares

Como múltiplos neurônios criam fronteiras complexas



Insight Central: Cada neurônio aprende UMA fronteira linear. A combinação de vários cria fronteiras NÃO lineares.

1 Perceptron (Linear)

Divide o espaço com UMA linha reta. Funciona para problemas simples: ex. separar clientes premium vs. não-premium por renda.



2 Combinação de 2+ neurônios

Cada neurônio contribui com uma região. A combinação delimita polígonos. Ex: "Renda alta E uso moderado → manter".

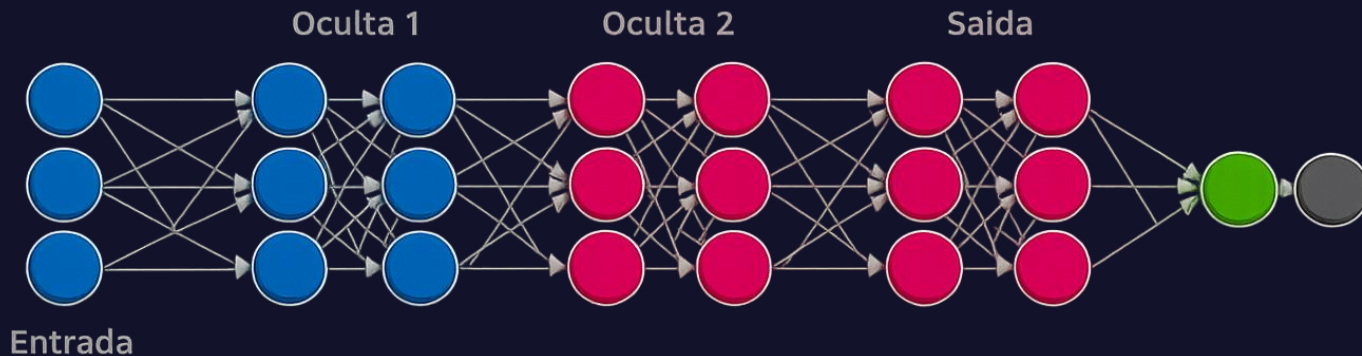


3 Rede profunda (Deep)

Múltiplas camadas aprendem hierarquias: dados brutos → padrões simples → padrões complexos → decisão.

Multi-Layer Perceptron (MLP)

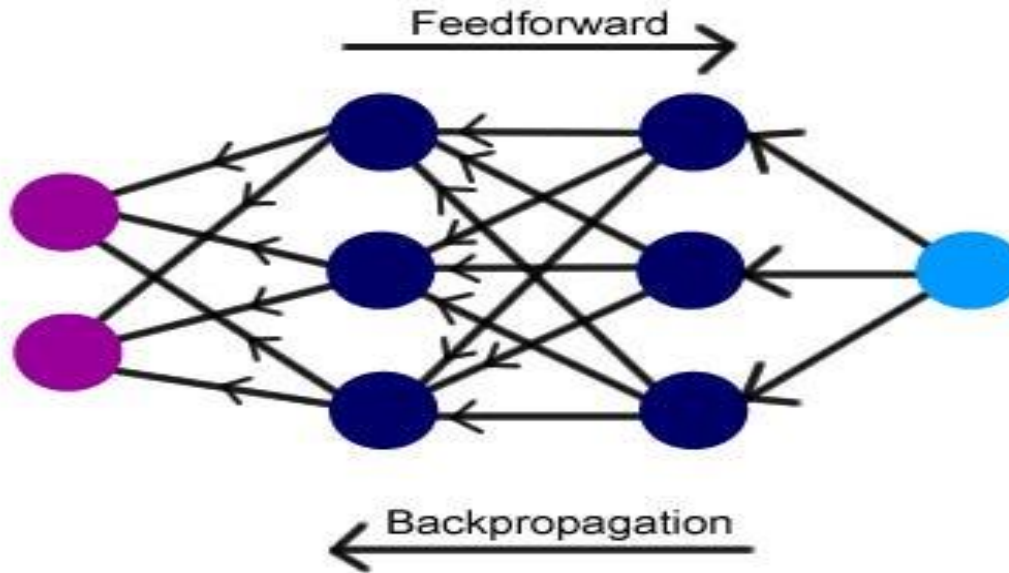
A arquitetura base de toda rede profunda



Camada de Entrada → raw features **Camadas Ocultas** → representações **Camada de Saída** → predição

Como funciona o fluxo da informação

A arquitetura base de toda rede profunda



Feedforward: como a informação flui

1

Entrada

Dados do cliente entram como vetor numérico:
 $x = [\text{Score}=720, \text{Renda}=8k, \text{Uso}=45\text{GB}]$

2

Soma Ponderada

Cada neurônio calcula:
 $z = \sum (w_i \cdot x_i) + b$
(multiplicação matricial por camada)

3

Ativação

Resultado passa pela função de ativação:
 $a = f(z) \rightarrow$ introduz não-linearidade

4

Propagação

A saída de uma camada vira a entrada da próxima. Repete-se até a camada de saída.

5

Predição

Última camada $\rightarrow$ probabilidade:
 $P(\text{churn}) = 0.82 \rightarrow \text{⚠ cliente em risco!}$



O feedforward é determinístico: mesmas entradas $\rightarrow$ mesma saída. O aprendizado acontece no backpropagation.



Backpropagation: como a rede aprende

Rumelhart, Hinton & Williams, 1986

💡 Intuição: Imagine um gerente (a rede) que fez uma previsão errada. O backprop é o "pós-mortem" que distribui culpa para cada funcionário (peso) proporcional ao seu impacto no erro.

① Forward Pass

Calcula $\hat{y}$ com os pesos atuais.
Rede prevê $P(\text{churn}) = 0.3$

② Calcula Perda

Compara $\hat{y}$ com y real:
 $L = -\log(0.3) = 1.2 \leftarrow$ ruim!

③ Gradiente ($\partial L / \partial w$)

Regra da cadeia calcula
contribuição de cada peso

④ Atualiza Pesos

$w \leftarrow w - \eta \cdot \partial L / \partial w$
 η = learning rate (0.001)

Chain Rule: o motor do Backpropagation

 Regra da Cadeia: "Como a mudança em W afeta o Erro passando por múltiplas funções compostas?"

Formalização:

$$\partial L / \partial w = \partial L / \partial \hat{y} \cdot \partial \hat{y} / \partial z \cdot \partial z / \partial w$$

$\partial L / \partial \hat{y}$ → gradiente da loss

$\partial \hat{y} / \partial z$ → derivada da ativação

$\partial z / \partial w$ → simplesmente $= x$

Exemplo Prático — Churn:

$$L = \text{BCE}(\hat{y}, y_{\text{real}})$$

$$\hat{y} = \text{sigmoid}(z)$$

$$z = w \cdot x_{\text{uso_dados}} + b$$

 A regra da cadeia permite calcular gradientes em QUALQUER profundidade de rede — isso é o que torna o deep learning escalável.

Funções de Ativação

Por que não podemos usar apenas somas lineares?

Sigmoid

$$\sigma(z) = 1 / (1 + e^{-z})$$

Saída: $[0, 1]$

✓ Interpretável como probabilidade

✗ Vanishing gradient em deep nets

ReLU

$$f(z) = \max(0, z)$$

Saída: $[0, \infty)$

✓ Rápido, sem vanishing gradient

✗ Dying ReLU (neurônios mortos)

Tanh

$$f(z) = (e^z - e^{-z}) / (e^z + e^{-z})$$

Saída: $[-1, 1]$

✓ Média zero → gradientes melhores

✗ Ainda sofre vanishing em extremos

Softmax

$$\sigma(z_i) = e^{z_i} / \sum e^{z_j}$$

Saída: probs somam 1

✓ Distribuição de probabilidade

✗ Só faz sentido na camada final



Learning Rate: o passo do aprendizado

O hiperparâmetro mais crítico do treinamento

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \partial \mathcal{L} / \partial \mathbf{w}$$



η muito alto

Passos gigantes $\rightarrow$ oscila em torno do mínimo $\rightarrow$ diverge
Ex: $\eta = 1.0$



η ideal

Converge suavemente para o mínimo
Ex: $\eta = 0.001$



η muito baixo

Passos minúsculos $\rightarrow$ demora demais ou para no mínimo local
Ex: $\eta = 0.000001$

Schedulers modernos: ReduceLROnPlateau · Cosine Annealing · Warm Restarts · Adam (lr adaptativo)

Overfitting & Underfitting

O dilema central do aprendizado de máquina

UNDERFITTING

Alta Bias / Baixa Variância

O modelo é simples demais para capturar os padrões.

Churn: rede rasa prevê todos os clientes como "vai ficar" (accuracy 78%, mas recall 0%!)

Train Acc: 60%

Test Acc: 59%

→ Aumentar profundidade

→ Mais neurônios

→ Treinar mais épocas

IDEAL

Bias e Variância Equilibrados

Generaliza bem: aprende padrões reais sem decorar os dados.

Churn: 85% AUC-ROC no conjunto de validação.

Train Acc: 88%

Test Acc: 86%

→ Validação cruzada

→ Early stopping

→ Tuning sistemático

OVERFITTING

Baixa Bias / Alta Variância

O modelo memorizou o treino. Em produção, falha com clientes novos.

Churn: 99% no treino, 62% no teste!

Train Acc: 99%

Test Acc: 62%

→ Dropout

→ Regularização L1/L2

→ Mais dados / augment.

Treino, Teste & Early Stopping

A divisão que garante generalização real

TREINO
70%

VALIDAÇÃO
15%

TESTE
15%

Treino (70%)

Ajuste dos pesos via backprop.
100k clientes históricos.

Validação (15%)

Monitoramento durante treino.
Early stopping aqui!

Teste (15%)

Avaliação final — NUNCA
visto durante treino!



Early Stopping — Parar na hora certa

Monitor: `val_loss` · Patience: 10 épocas · Restaurar: melhores pesos → Evita overfitting automaticamente!



Salve sempre o modelo com menor `val_loss` — não o da última época!



Regularização L1 & L2

Penalizar pesos grandes para evitar overfitting

L1 — LASSO

$$L_{total} = L + \lambda \cdot \sum |w_i|$$

- ✓ Produz pesos ESPARSOS (zeros)
- ✓ Seleção implícita de features
- Churn: elimina features irrelevantes (ex: cor favorita do cliente)
- ★ Ideal: muitas features, poucas relevantes

L2 — Ridge

$$L_{total} = L + \lambda \cdot \sum w_i^2$$

- ✓ Pesos pequenos mas NÃO zero
- ✓ Distribui importância entre features
- Churn: usa todas features mas penaliza features de ruído
- ★ Ideal: todas features relevantes

λ (lambda) controla a força da penalização. $\lambda \rightarrow 0$: sem regularização · $\lambda \rightarrow \infty$: pesos $\rightarrow 0$. Tunar via cross-validation!

Dropout: apagando neurônios aleatoriamente

Srivastava et al., 2014 — uma das técnicas mais eficazes



Analogia de Negócio: Numa empresa, se SEMPRE as mesmas pessoas tomam todas as decisões, a empresa fica dependente delas. Drop out força cada funcionário (neurônio) a aprender a trabalhar de forma independente.



Durante treino

p% dos neurônios são desligados aleatoriamente em cada batch.
Força redundância e robustez.



Durante teste

Todos os neurônios ativos.
Pesos escalados por (1-p).
Comporta-se como ensemble!



Valores típicos

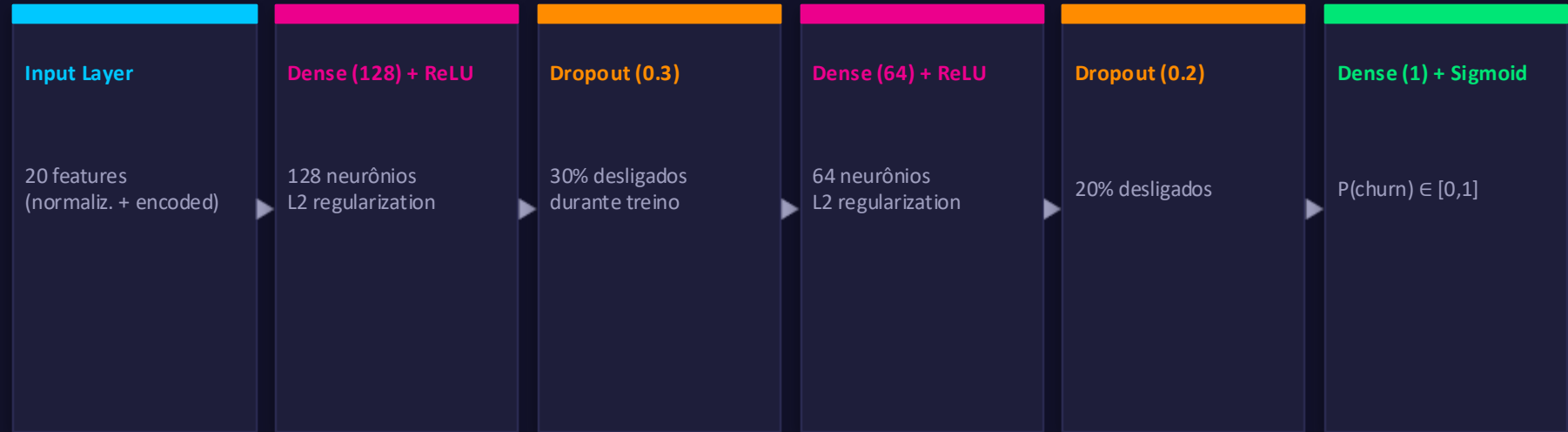
p = 0.2 a 0.5
Dense layers: 0.3-0.5
Conv layers: 0.1-0.2

```
model.add(Dense(128, activation='relu'))    model.add(Dropout(0.3))    # 30% dropout
```



Arquitetura Completa: Churn Prediction

Juntando todos os conceitos da aula



Configuração de Treino:

Optimizer: Adam
(lr=0.001)

Loss: Binary
Crossentropy

Batch Size: 256

Epochs: 100 (early stop
p=10)

Class Weight: balanced
(churn raro!)

O que vem pela frente

Roadmap completo — Feature Engineering + Deep Learning

Aula 01 **Fundamentos: Redes Neurais, Backprop, Regularização**

← *você está aqui*

Aula 02 CNNs: Visão Computacional & Reconhecimento de Imagens

Aula 03 RNNs / LSTMs: Séries Temporais & NLP

Aula 04 Transfer Learning & Fine-Tuning

Aula 05 Transformers & Atenção (base dos LLMs)

Aula 06 Deploy & MLOps: da Rede ao Produto

Próximos Passos



Notebook: implemente a MLP de churn com Keras



Experimente: troque ReLU por Tanh — o que muda?



Análise: varie o learning rate e plote as curvas de loss



Reflexão: quando L1 é preferível a Dropout?

